



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления (ИУ)»

КАФЕДРА «Информационные системы и телекоммуникации (ИУЗ)»

ОТЧЕТ

по лабораторной работе №2
по курсу «Разработка Программного обеспечения»
на тему: «Разработка REST API сервера авторизации платежей
транспортными картами»

Студент ИУЗ-61Б
(Группа)

(Подпись, дата)

Пялкин А. А.
(И. О. Фамилия)

Преподаватель

(Подпись, дата)

Сидякин И.М.
(И. О. Фамилия)

2026 г.

1 Теоретическая часть

1.1 Цель работы

Целью лабораторной работы является разработка REST API сервера авторизации платежей транспортными картами. Сервер должен принимать запросы от платежных терминалов, проверять данные транспортной карты, выполнять авторизацию платежной транзакции и сохранять информацию о выполненной операции в базе данных.

1.2 Постановка задачи

В ходе выполнения лабораторной работы необходимо разработать серверное приложение на языке Go. В качестве системы хранения данных используется SQLite3. Приложение должно запускаться из Docker-контейнера, а доступ к REST API должен выполняться через веб-сервер Nginx по протоколу HTTPS.

В системе должны быть реализованы таблицы пользователей, терминалов, транспортных карт MIFARE, ключей и транзакций. Для управления структурой базы данных необходимо использовать систему миграций goose.

Также требуется реализовать вход пользователя по логину и паролю, авторизацию с помощью JWT, CRUD API для редактирования таблиц базы данных, API терминала для авторизации платежей и загрузки ключей, а также страницу Swagger-документации.

1.3 Используемые технологии

Для реализации серверной части используется язык Go. Он применяется для обработки HTTP-запросов, работы с базой данных, проверки прав доступа и выполнения бизнес-логики авторизации платежа.

Для хранения данных используется SQLite3. Эта база данных подходит для лабораторного проекта, так как не требует отдельного серверного процесса и хранит данные в одном файле.

REST API используется для взаимодействия клиента и сервера по HTTP. Для работы с ресурсами применяются методы GET, POST, PUT и DELETE. Ответы сервера передаются в формате JSON.

Авторизация пользователей реализована с помощью JWT-токенов. После

успешного входа пользователь получает токен и передает его в заголовке Authorization при обращении к защищенным маршрутам.

Docker используется для запуска приложения в изолированном окружении. Nginx выполняет роль обратного прокси: принимает HTTPS-запросы на порту 8888 и перенаправляет их на Go-сервер. Swagger используется для документирования и проверки REST API через браузер.

2 Практическая часть

2.1 Общая схема решения

Разработанная система состоит из клиента, веб-сервера Nginx, backend-сервера на языке Go и базы данных SQLite3.

Клиент отправляет HTTPS-запросы на адрес `https://localhost:8888/api/v1`. Эти запросы принимает Nginx, который выполняет роль обратного прокси и перенаправляет их на Go-сервер. Backend-сервер обрабатывает REST API-запросы, выполняет авторизацию пользователей, проверяет права доступа, работает с базой данных и возвращает клиенту ответ в формате JSON.

В качестве клиента при проверке лабораторной работы использовались Swagger UI и HTTP-запросы. Отдельный графический frontend в рамках данной лабораторной работы не разрабатывался, так как основная цель работы заключалась в реализации REST API сервера авторизации платежей.

Общая схема работы системы:

```
Client / Swagger / Terminal request
      |
      | HTTPS :8888
      v
Nginx reverse proxy
      |
      | HTTP :8080
      v
Go REST API server
      |
      v
SQLite database
```

2.2 Структура проекта

В ходе выполнения лабораторной работы был разработан сервер авторизации платежей транспортными картами. Проект содержит исходный код Go-сервера, миграции базы данных, конфигурацию Nginx, Dockerfile, docker-compose.yml и файлы Swagger-документации.

Основная структура проекта:

```
transport_api/  
|-- cmd/  
|-- data/  
|-- docs/  
|-- internal/  
|   |-- config/  
|   |-- db/  
|   |-- handlers/  
|   |-- middleware/  
|   |-- models/  
|   |-- repositories/  
|   '-- services/  
|-- migrations/  
|-- nginx/  
|-- .dockerignore  
|-- docker-compose.yml  
|-- docker-entrypoint.sh  
|-- Dockerfile  
|-- go.mod  
'-- go.sum
```

В каталоге **cmd** находится точка входа приложения. Каталог **internal** содержит основную backend-логику: конфигурацию, подключение к базе данных, обработчики запросов, **middleware**, модели, репозитории и сервисы. Каталог **migrations** содержит SQL-миграции, каталог **nginx** — конфигурацию веб-сервера, а каталог **docs** — файлы Swagger-документации.

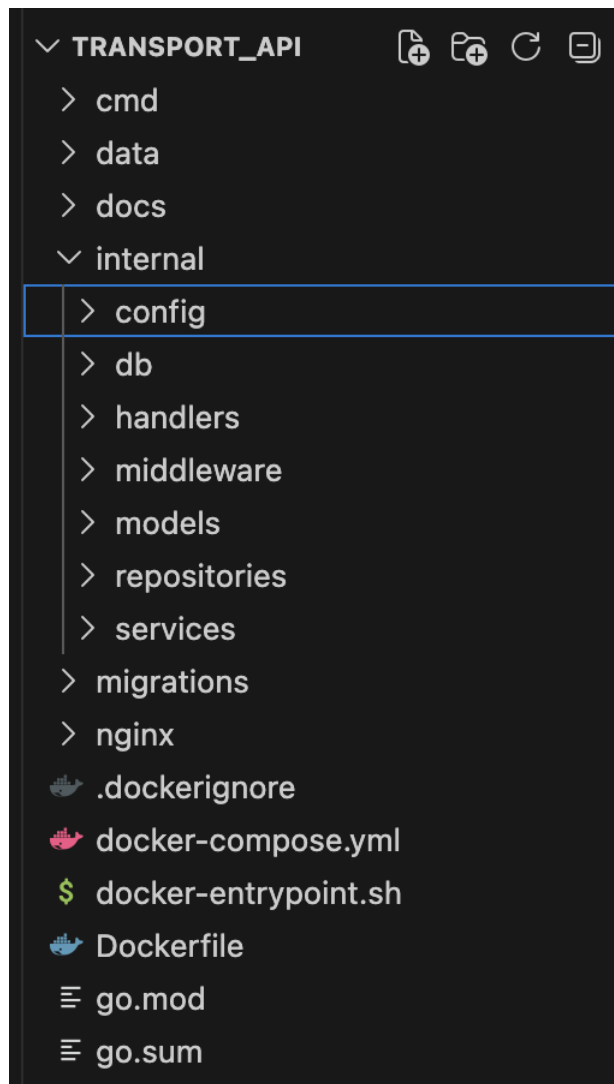


Рисунок 2.1 – Структура проекта сервера авторизации платежей

2.3 Структура базы данных

Для хранения данных используется SQLite3. В базе данных были созданы таблицы пользователей, терминалов, транспортных карт, ключей и транзакций.

Таблица пользователей хранит логин, имя пользователя, хеш пароля и роль. Роль определяет права доступа пользователя: обычный пользователь имеет ограниченный доступ, а администратор может выполнять операции создания, изменения и удаления данных.

Таблица терминалов хранит сведения о платежных терминалах: серийный номер, название и адрес установки. Таблица транспортных карт хранит номер карты, баланс, флаг блокировки, имя владельца и ссылку на ключ. Таблица ключей содержит ключи MIFARE, которые могут быть связаны с

несколькими картами. Таблица транзакций хранит историю платежей, сумму операции, дату и время, ссылку на карту и ссылку на терминал.

```
-- +goose Up
CREATE TABLE IF NOT EXISTS users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  login TEXT UNIQUE NOT NULL,
  name TEXT NOT NULL,
  password_hash TEXT NOT NULL,
  is_admin BOOLEAN DEFAULT 0,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS terminals (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  serial_number TEXT UNIQUE NOT NULL,
  address TEXT,
  name TEXT,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS keys (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  key_value TEXT NOT NULL,
  description TEXT,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS cards (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  card_number TEXT UNIQUE NOT NULL,
  balance REAL DEFAULT 0.0,
  is_locked BOOLEAN DEFAULT 0,
  owner_name TEXT,
  key_id INTEGER,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (key_id) REFERENCES keys(id)
);

CREATE TABLE IF NOT EXISTS transactions (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  amount REAL NOT NULL,
  card_id INTEGER NOT NULL,
  terminal_id INTEGER NOT NULL,
  transaction_date DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (card_id) REFERENCES cards(id),
  FOREIGN KEY (terminal_id) REFERENCES terminals(id)
);
```

Рисунок 2.2 – Структура таблиц базы данных SQLite

2.4 Backend

Backend-часть реализована на языке Go. Сервер обрабатывает HTTP-запросы, выполняет авторизацию пользователей, проверяет JWT-токены, реализует CRUD API для таблиц базы данных и предоставляет специальные маршруты для платежного терминала.

Основная бизнес-логика связана с авторизацией платежной транзакции. При получении запроса сервер проверяет существование терминала, наличие карты, флаг блокировки карты и достаточность средств на балансе. После успешной проверки создается запись о транзакции, а баланс карты уменьшается на сумму платежа.

Для доступа к базе данных используется слой репозитория. Он отделяет SQL-запросы от обработчиков REST API и делает структуру проекта более удобной для поддержки.

2.5 Авторизация и разграничение прав

Для входа пользователя в систему реализован маршрут:

```
POST /api/v1/auth/login
```

Пользователь отправляет логин и пароль. Сервер проверяет наличие пользователя в базе данных и сравнивает пароль с сохраненным хешем. При успешной проверке сервер формирует JWT-токен и возвращает его клиенту.

Полученный токен используется для доступа к защищенным маршрутам:

```
Authorization: Bearer <JWT_TOKEN>
```

В системе реализованы две роли: пользователь и администратор. Проверка токена выполняется с помощью middleware. Если токен отсутствует, некорректен или истек, сервер возвращает ошибку авторизации. Администратор имеет права на создание, изменение и удаление данных, а обычный пользователь имеет ограниченный доступ.



Рисунок 2.3 – Получение JWT-токена после авторизации пользователя

2.6 CRUD API

Для основных таблиц базы данных были реализованы CRUD-операции. CRUD включает создание, чтение, обновление и удаление записей.

Основные CRUD-маршруты имеют следующую структуру:

```
GET, POST, PUT, DELETE /api/v1/users
GET, POST, PUT, DELETE /api/v1/terminals
GET, POST, PUT, DELETE /api/v1/cards
GET, POST, PUT, DELETE /api/v1/keys
```


Для транзакций реализовано получение истории операций. Создание транзакции выполняется через специальный терминальный маршрут авторизации платежа, так как при создании транзакции требуется проверить карту, терминал, блокировку и баланс.

2.7 API терминала

Для платежного терминала были реализованы два основных маршрута: авторизация платежной транзакции и загрузка ключей.

Маршрут авторизации платежной транзакции:

```
POST /api/v1/terminal/transactions/authorize
```

Терминал отправляет на сервер серийный номер терминала, номер карты и сумму операции. Сервер проверяет существование терминала, существование карты, флаг блокировки карты и достаточность средств на балансе.

Если карта существует, не заблокирована, а средств достаточно, сервер создает запись о транзакции и уменьшает баланс карты на сумму операции. После этого терминалу возвращается результат успешной авторизации. Если одна из проверок не пройдена, сервер возвращает отказ в выполнении платежа.

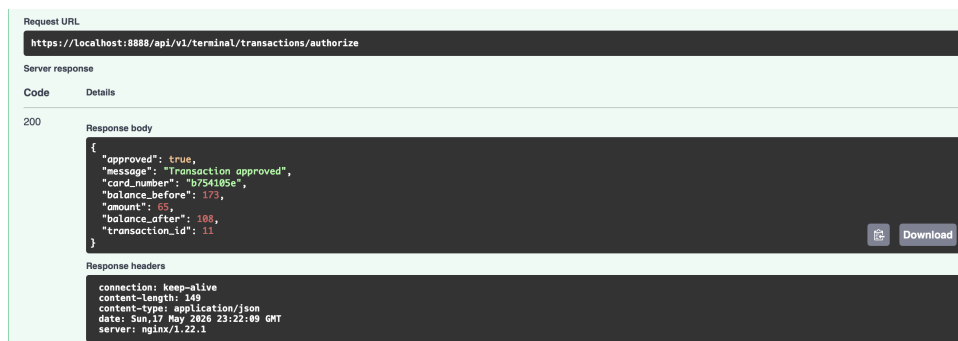


Рисунок 2.4 – Авторизация платежной транзакции транспортной картой

Маршрут загрузки ключей:

```
GET /api/v1/terminal/keys?terminal_serial=TERM-001
```

Данный маршрут используется терминалом для получения ключей, необходимых для работы с транспортными картами.

2.8 Frontend и клиент

Отдельное frontend-приложение в рамках лабораторной работы №2 не разрабатывалось. Для проверки и демонстрации REST API использовалась

Swagger-документация, доступная через браузер. Swagger UI позволяет просматривать маршруты API, структуру запросов и ответов, а также выполнять тестовые запросы к серверу.

Клиент платежного терминала в рамках данной лабораторной работы был представлен HTTP-запросами к REST API. Терминал отправляет на сервер номер карты, сумму платежа и серийный номер терминала. Сервер выполняет проверку данных и возвращает результат авторизации платежа.

Flutter-клиент в данной лабораторной работе не реализовывался, так как лабораторная работа №2 посвящена backend-серверу авторизации платежей. Возможность подключения Flutter-клиента может быть реализована в следующих лабораторных работах через уже созданные REST API маршруты.

2.9 Docker и Nginx

Для запуска приложения был подготовлен Dockerfile. Сборка выполняется с использованием многоэтапного подхода. На первом этапе выполняется сборка Go-приложения, а на втором этапе создается финальный образ, содержащий исполняемый файл сервера, базу данных, Nginx и необходимые конфигурационные файлы.

Nginx настроен как обратный прокси. Он принимает HTTPS-запросы на порту 8888 и перенаправляет их на Go-сервер. Для работы HTTPS используются TLS-ключи и сертификаты.

Запуск контейнеров выполняется командой:

```
docker compose up --build
```

```
aleksandr@palkinMacBook-Pro-4 transport_api % docker compose up --build
=> resolve image config for docker-image://docker.io/docker/dockerfile:1 16.2s
=> CACHED docker-image://docker.io/docker/dockerfile:1@sha256:2780b5c3bab67f1f76c781860de469442999ed1a0d7992a5efdf2cffc0e3d 0.0s
=> resolve docker.io/docker/dockerfile:1@sha256:2780b5c3bab67f1f76c781860de469442999ed1a0d7992a5efdf2cffc0e3d769 0.0s
=> [internal] load metadata for docker.io/library/debian:bookworm-slim 0.7s
=> [internal] load metadata for docker.io/library/golang:1.26-bookworm 0.7s
=> [internal] load .dockerignore 0.0s
=> transferring context: 83B 0.0s
=> [builder 1/6] FROM docker.io/library/golang:1.26-bookworm@sha256:252599aeb51ad60b83e4d8821802068127c528c707cb7dd7af9d93be 0.0s
=> resolve docker.io/library/golang:1.26-bookworm@sha256:252599aeb51ad60b83e4d8821802068127c528c707cb7dd7af9d93be057c6011 0.0s
=> [stage-1 1/10] FROM docker.io/library/debian:bookworm-slim@sha256:67b30a61dc87758f0caf819646104f29ecbda97d920aaf5edc834128ac8493d 0.0s
=> resolve docker.io/library/debian:bookworm-slim@sha256:67b30a61dc87758f0caf819646104f29ecbda97d920aaf5edc834128ac8493d 0.0s
=> [internal] load build context 0.0s
=> transferring context: 2.53kB 0.0s
=> CACHED [stage-1 2/10] RUN apt-get update && apt-get install -y --no-install-recommends nginx ca-certificates && 0.0s
=> CACHED [stage-1 3/10] WORKDIR /app 0.0s
=> CACHED [builder 2/6] WORKDIR /src 0.0s
=> CACHED [builder 3/6] COPY go.mod go.sum ./ 0.0s
=> CACHED [builder 4/6] RUN go mod download 0.0s
=> CACHED [builder 5/6] COPY . . 0.0s
=> CACHED [builder 6/6] RUN CGO_ENABLED=0 GOOS=linux go build -o /out/transport_api ./cmd/api 0.0s
=> CACHED [stage-1 4/10] COPY --from=builder /out/transport_api /app/transport_api 0.0s
=> CACHED [stage-1 5/10] COPY migrations /app/migrations 0.0s
=> CACHED [stage-1 6/10] COPY docs /app/docs 0.0s
=> CACHED [stage-1 7/10] COPY nginx/nginx.conf /etc/nginx/nginx.conf 0.0s
=> CACHED [stage-1 8/10] COPY nginx/certs /etc/nginx/certs 0.0s
=> CACHED [stage-1 9/10] COPY docker-entrypoint.sh /app/docker-entrypoint.sh 0.0s
=> CACHED [stage-1 10/10] RUN mkdir -p /app/data && chmod +x /app/docker-entrypoint.sh 0.0s
=> exporting to image 0.0s
=> exporting layers 0.0s
=> exporting manifest sha256:20de2cc4904557c82b850a9997e51dae5d9ec06327cc23bd49d7e860ba2f3391 0.0s
=> exporting config sha256:b42f78321300fc468c8e392d47e83faa759964573dff65098af8b33b6a26e 0.0s
=> exporting attestation manifest sha256:8eb77b5008c9f2018901d834268b8fabef09ac4eaa9519a8d8ecb4a20823df1d 0.0s
=> exporting manifest list sha256:dfd2465e10b72b92fa0c193ad6a1910c2f43e0e5a3dbbca9acb556559f0e6156 0.0s
=> naming to docker.io/library/transport_api-transport-api:latest 0.0s
=> unpacking to docker.io/library/transport_api-transport-api:latest 0.0s
=> resolving provenance for metadata file 0.0s
[+] up 3/3
✓ Image transport_api-transport-api Built 17.4s
✓ Network transport_api_default Created 0.0s
✓ Container transport_api Created 0.0s
Attaching to transport_api
transport_api | Starting transport_api...
transport_api | Starting nginx...
transport_api | 2026/05/18 00:04:52 goose: no migrations to run. current version: 6
transport_api | 2026/05/18 00:04:52 transport_api listening on :8080
```

Рисунок 2.5 – Запуск сервера и Nginx в Docker-контейнере

2.10 Swagger-документация

Для разработанного REST API была подготовлена Swagger-документация. Она доступна из Docker-контейнера по адресу:

```
https://localhost:8888/api/v1/swagger
```

Swagger-документация содержит описание маршрутов, параметров запросов и структуры ответов. С ее помощью можно проверить работу API через браузер.

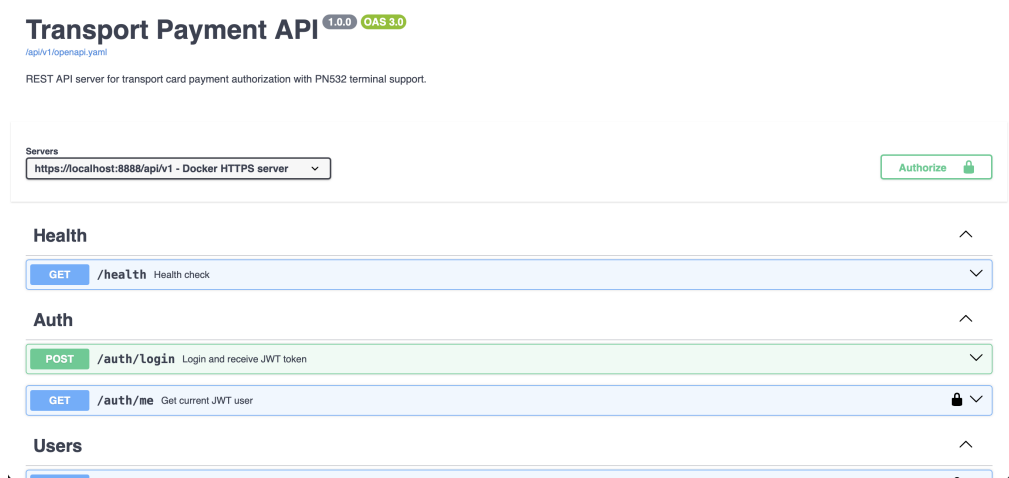


Рисунок 2.6 – Swagger-документация REST API

2.11 Вывод

В ходе выполнения лабораторной работы был разработан REST API сервер авторизации платежей транспортными картами. Были реализованы база данных SQLite3, миграции goose, JWT-авторизация, CRUD API для основных таблиц, API терминала для авторизации платежа и загрузки ключей, Swagger-документация, а также запуск приложения в Docker-контейнере через Nginx по протоколу HTTPS.

3 Приложение

3.1 История транзакций

После успешной авторизации платежа сервер создает запись в таблице транзакций. В записи сохраняется сумма операции, идентификатор карты, идентификатор терминала, дата и время выполнения операции, а также результат авторизации.

На рисунке 3.1 представлена история выполненных транзакций.



Рисунок 3.1 – История выполненных транзакций

3.2 Основные маршруты REST API

В разработанном сервере были реализованы следующие основные маршруты:

```
GET    /api/v1/health

POST   /api/v1/auth/login
GET    /api/v1/auth/me

GET     /api/v1/users
POST    /api/v1/users
GET     /api/v1/users/{id}
PUT     /api/v1/users/{id}
DELETE  /api/v1/users/{id}

GET     /api/v1/terminals
POST    /api/v1/terminals
GET     /api/v1/terminals/{id}
```

```
PUT      /api/v1/terminals/{id}
DELETE   /api/v1/terminals/{id}

GET       /api/v1/cards
POST      /api/v1/cards
GET       /api/v1/cards/{id}
PUT       /api/v1/cards/{id}
DELETE    /api/v1/cards/{id}

GET       /api/v1/keys
POST      /api/v1/keys
GET       /api/v1/keys/{id}
PUT       /api/v1/keys/{id}
DELETE    /api/v1/keys/{id}

GET       /api/v1/transactions
POST      /api/v1/terminal/transactions/authorize
GET       /api/v1/terminal/keys

GET       /api/v1/swagger
```

4 Список литературы

1. Документация языка Go. Режим доступа: <https://go.dev/doc/>
2. Документация SQLite. Режим доступа: <https://www.sqlite.org/docs.html>
3. Документация Docker. Режим доступа: <https://docs.docker.com/>
4. Документация Nginx. Режим доступа: <https://nginx.org/en/docs/>
5. Документация goose. Режим доступа: <https://github.com/pressly/goose>
6. Документация OpenAPI/Swagger. Режим доступа: <https://swagger.io/docs/>
7. RFC 7519: JSON Web Token. Режим доступа: <https://www.rfc-editor.org/rfc/rfc7519>